

Guía ampliada de clase

Funciones en Python, matrices, entypoint y buenas prácticas de programación modular

Propósito	Servir como material de estudio para comprender, diseñar y organizar programas usando funciones en Python.
Curso	Fundamentos de Programación Imperativa
Nivel	Primer semestre de Ingeniería de Sistemas
Enfoque	Comprender la función como herramienta para dividir problemas, reutilizar lógica y controlar la ejecución del programa.

Idea central: una función no existe solo para escribir menos código; existe para darle estructura, claridad y reutilización a la solución de un problema.

1. ¿Por qué aparecen las funciones?

Hasta este punto del curso, muchos programas se pueden resolver escribiendo instrucciones de arriba hacia abajo. Sin embargo, a medida que los problemas crecen, aparecen tres dificultades:

- Repetición: el mismo bloque de código se copia varias veces.
- Desorden: entrada, procesamiento y salida quedan mezclados.
- Dificultad para corregir: si hay un error, se vuelve más difícil saber dónde está.

Las funciones permiten enfrentar estos problemas dividiendo el programa en partes más pequeñas y con responsabilidad clara.

1.1 Ejemplo de código repetido

```
nota1 = float(input("Ingrese nota 1: "))
nota2 = float(input("Ingrese nota 2: "))
nota3 = float(input("Ingrese nota 3: "))

promedio = (nota1 + nota2 + nota3) / 3
print("Promedio:", promedio)

nota1 = float(input("Ingrese nota 1: "))
nota2 = float(input("Ingrese nota 2: "))
nota3 = float(input("Ingrese nota 3: "))

promedio = (nota1 + nota2 + nota3) / 3
print("Promedio:", promedio)
```

El problema no es que el código sea incorrecto, sino que se repite. Si la fórmula del promedio cambia, habría que modificarla en muchas partes. Una función evita esa duplicación.

2. Concepto de función

Una función es un bloque de código con nombre que realiza una tarea específica. Puede recibir datos, procesarlos y devolver un resultado.

Parte	Significado
Nombre	Identifica la función. Debe describir lo que hace.
Parámetros	Datos que la función necesita para trabajar.
Cuerpo	Instrucciones que se ejecutan cuando la función es llamada.

Return	Valor que la función devuelve al lugar donde fue llamada.
--------	---

2.1 Estructura básica

```
def nombre_funcion(parametros):
    instrucciones
    return resultado
```

2.2 Primer ejemplo

```
def saludar():
    print("Hola, bienvenido")

saludar()
```

Definir una función no significa ejecutarla. Una función solo se ejecuta cuando se llama por su nombre.

3. Parámetros y argumentos

Los parámetros son las variables que aparecen en la definición de la función. Los argumentos son los valores concretos que se envían cuando la función se llama.

```
def cuadrado(numero):          # numero es un parámetro
    return numero * numero

resultado = cuadrado(5)        # 5 es un argumento
print(resultado)               # 25
```

3.1 Función con varios parámetros

```
def calcular_promedio(a, b, c):
    return (a + b + c) / 3

promedio = calcular_promedio(4.0, 3.5, 5.0)
print(promedio)
```

3.2 Error frecuente: cambiar el orden de los argumentos

```
def dividir(a, b):
    return a / b

print(dividir(10, 2)) # 5.0
print(dividir(2, 10)) # 0.2
```

Cuando una función tiene varios parámetros, el orden de los argumentos importa. No siempre cambiar el orden genera error; a veces genera un resultado lógico incorrecto.

4. print() vs return

Este es uno de los puntos más importantes. print() muestra un valor en pantalla; return devuelve un valor para que pueda usarse después.

print()	return
Muestra información al usuario.	Entrega un valor al programa.
No permite reutilizar fácilmente el resultado.	Permite guardar el resultado en una variable.
Se usa para salida.	Se usa para procesamiento.

4.1 Comparación

```
def suma_con_print(a, b):
    print(a + b)

def suma_con_return(a, b):
```

```
    return a + b

x = suma_con_print(2, 3)
print(x) # None

y = suma_con_return(2, 3)
print(y) # 5
```

Regla práctica: si el resultado será usado por otra parte del programa, la función debe usar **return**.

5. Funciones y prueba de escritorio

Las funciones también se pueden analizar con prueba de escritorio. Para hacerlo, se debe identificar:

- Qué valores entran como argumentos.
- Qué valores toman los parámetros.
- Qué instrucciones se ejecutan.
- Qué valor retorna la función.
- Dónde se usa ese valor retornado.

5.1 Ejemplo de seguimiento

```
def doble(x):
    return x * 2

a = 5
b = doble(a)
c = doble(b)

print(c)
```

Prueba de escritorio: $a = 5$; $b = \text{doble}(5) = 10$; $c = \text{doble}(10) = 20$. Por tanto, la salida es 20.

6. Funciones auxiliares

Una función auxiliar es una función pequeña que ayuda a resolver una parte del problema. Permite que el programa principal sea más claro.

6.1 Ejemplo sin función auxiliar

```
def contar_vocales(texto):
    contador = 0

    for letra in texto:
        if letra in "aeiou":
            contador += 1

    return contador
```

6.2 Ejemplo con función auxiliar

```
def es_vocal(letra):
    return letra in "aeiou"

def contar_vocales(texto):
    contador = 0

    for letra in texto:
        if es_vocal(letra):
            contador += 1

    return contador
```

La función `es_vocal` permite separar la condición del recorrido. Esto mejora la lectura del programa.

6.3 Buenas funciones auxiliares

- Tienen nombres claros.
- Realizan una sola tarea.
- Devuelven un resultado cuando corresponde.
- Son fáciles de probar con ejemplos pequeños.
- Evitan repetir condiciones complejas.

7. Funciones con listas

Las listas suelen combinarse muy bien con funciones, porque muchas tareas consisten en recorrer datos y devolver un resultado.

7.1 Contar pares

```
def es_par(numero):
    return numero % 2 == 0

def contar_pares(lista):
    contador = 0

    for numero in lista:
        if es_par(numero):
            contador += 1

    return contador

datos = [1, 2, 3, 4, 5, 6]
print(contar_pares(datos)) # 3
```

7.2 Buscar el mayor

```
def mayor_lista(lista):
    mayor = lista[0]

    for elemento in lista:
        if elemento > mayor:
            mayor = elemento

    return mayor

print(mayor_lista([4, 9, 2, 7])) # 9
```

Este ejemplo asume que la lista tiene al menos un elemento. En programas más robustos se debe validar si la lista está vacía.

7.3 Retornar una nueva lista

```
def filtrar_positivos(lista):
    positivos = []

    for elemento in lista:
        if elemento > 0:
            positivos.append(elemento)

    return positivos

print(filtrar_positivos([3, -1, 0, 5, -2]))
# [3, 5]
```

8. Funciones con cadenas

Las cadenas pueden recorrerse carácter por carácter, de forma similar a una lista.

8.1 Contar una letra

```
def contar_letra(texto, objetivo):
    contador = 0

    for letra in texto:
        if letra == objetivo:
            contador += 1

    return contador

print(contar_letra("programacion", "o")) # 2
```

8.2 Invertir una cadena

```
def invertir(texto):
    resultado = ""

    for letra in texto:
        resultado = letra + resultado

    return resultado

print(invertir("hola")) # aloh
```

8.3 Normalizar texto

```
def normalizar(texto):
    return texto.lower().strip()

print(normalizar(" HoLa ")) # hola
```

Normalizar significa transformar los datos a un formato común para facilitar comparaciones. Por ejemplo, convertir todo a minúsculas.

9. Funciones con matrices

Una matriz puede representarse como una lista de listas. Las funciones permiten encapsular recorridos y operaciones sobre matrices.

```
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

9.1 Sumar todos los elementos

```
def suma_matriz(matriz):
    suma = 0

    for fila in matriz:
        for elemento in fila:
            suma += elemento

    return suma

print(suma_matriz(matriz)) # 45
```

9.2 Contar positivos

```
def contar_positivos_matriz(matriz):
    contador = 0

    for fila in matriz:
        for elemento in fila:
            if elemento > 0:
                contador += 1

    return contador
```

9.3 Buscar el mayor elemento

```
def mayor_matriz(matriz):
    mayor = matriz[0][0]

    for fila in matriz:
        for elemento in fila:
            if elemento > mayor:
                mayor = elemento

    return mayor
```

9.4 Sumar una fila específica

```
def sumar_fila(matriz, indice_fila):
    suma = 0

    for elemento in matriz[indice_fila]:
        suma += elemento

    return suma

m = [
    [1, 2, 3],
    [4, 5, 6]
]

print(sumar_fila(m, 1)) # 15
```

9.5 Contar elementos que cumplen una condición

```
def es_mayor_que_cinco(numero):
    return numero > 5

def contar_mayores_que_cinco(matriz):
    contador = 0

    for fila in matriz:
        for elemento in fila:
            if es_mayor_que_cinco(elemento):
                contador += 1

    return contador
```

Las funciones con matrices son una forma organizada de reutilizar patrones clásicos: sumar, contar, buscar, filtrar o transformar datos.

10. El entrypoint en Python

El entrypoint es el punto de entrada del programa. En Python se suele controlar con:

```
if __name__ == "__main__":  
    # Código principal
```

10.1 ¿Qué significa `__name__`?

`__name__` es una variable especial que Python asigna automáticamente a cada archivo.

- Si el archivo se ejecuta directamente, `__name__` vale `'__main__'`.
- Si el archivo se importa desde otro archivo, `__name__` toma el nombre del módulo.

10.2 Ejemplo sin entrypoint

```
def saludar():  
    print("Hola")
```

```
saludar()
```

En este caso, la llamada a `saludar()` se ejecuta directamente al ejecutar el archivo. En programas pequeños funciona, pero en programas más grandes conviene controlar mejor el flujo.

10.3 Ejemplo con entrypoint

```
def saludar():  
    print("Hola")
```

```
if __name__ == "__main__":  
    saludar()
```

La función queda definida, pero el bloque principal solo se ejecuta cuando el archivo se usa como programa principal.

10.4 Uso recomendado con `main()`

```
def leer_numero(mensaje):  
    return int(input(mensaje))
```

```
def sumar(a, b):  
    return a + b
```

```
def main():  
    n1 = leer_numero("Ingrese el primer número: ")  
    n2 = leer_numero("Ingrese el segundo número: ")  
    resultado = sumar(n1, n2)  
    print("Resultado:", resultado)
```

```
if __name__ == "__main__":  
    main()
```

La función `main()` actúa como organizadora del programa. Allí se coordina la entrada, el procesamiento y la salida.

10.5 ¿Por qué usar entrypoint?

- Permite separar definición de funciones y ejecución del programa.
- Evita que ciertas instrucciones se ejecuten automáticamente cuando el archivo se importa.
- Mejora la organización del código.
- Facilita pruebas y reutilización.
- Acostumbra al estudiante a estructurar programas más grandes.

11. Organización funcional del programa

Aunque Python no obliga a escribir todo con funciones, es una buena práctica organizar el código en partes.

11.1 Separar responsabilidades

Responsabilidad	Ejemplo de función
Leer datos	leer_numero(), leer_lista()
Procesar datos	calcular_promedio(), contar_pares()
Validar condiciones	es_par(), es_positivo()
Mostrar resultados	mostrar_resultado()
Coordinar el programa	main()

11.2 Ejemplo de programa organizado

```
def es_aprobado(nota):
    return nota >= 3.0

def contar_aprobados(notas):
    contador = 0

    for nota in notas:
        if es_aprobado(nota):
            contador += 1

    return contador

def main():
    notas = [4.5, 2.8, 3.0, 5.0]
    aprobados = contar_aprobados(notas)
    print("Aprobados:", aprobados)

if __name__ == "__main__":
    main()
```

Este programa es fácil de leer porque cada función tiene una responsabilidad clara.

12. Buenas prácticas al programar con funciones

- Usar nombres descriptivos: contar_pares es mejor que cp.
- Una función, una tarea: si una función lee datos, calcula y además imprime, probablemente hace demasiado.
- Preferir return para resultados: imprimir dentro de una función solo cuando su responsabilidad sea mostrar.
- Evitar repetir código: si una lógica aparece varias veces, probablemente merece una función.
- Probar funciones pequeñas: antes de integrarlas al programa completo.
- Evitar variables globales: pasar datos como parámetros y retornar resultados.
- Usar main(): para coordinar el flujo principal.
- Usar entrypoint: para controlar la ejecución del archivo.

12.1 Mal ejemplo: función con demasiadas responsabilidades

```
def proceso():
    notas = [4.5, 2.8, 3.0]
    contador = 0

    for nota in notas:
```

```
        if nota >= 3.0:
            contador += 1

    print("Aprobados:", contador)
```

12.2 Mejor ejemplo: responsabilidades separadas

```
def es_aprobado(nota):
    return nota >= 3.0

def contar_aprobados(notas):
    contador = 0

    for nota in notas:
        if es_aprobado(nota):
            contador += 1

    return contador

def mostrar_aprobados(cantidad):
    print("Aprobados:", cantidad)

def main():
    notas = [4.5, 2.8, 3.0]
    cantidad = contar_aprobados(notas)
    mostrar_aprobados(cantidad)

if __name__ == "__main__":
    main()
```

13. Errores comunes

13.1 Definir una función y no llamarla

```
def saludar():  
    print("Hola")  
  
# No se imprime nada porque la función no fue llamada.
```

Solución:

```
saludar()
```

13.2 Usar print cuando se necesita return

```
def sumar(a, b):  
    print(a + b)  
  
resultado = sumar(2, 3)  
print(resultado) # None
```

Si se necesita guardar el resultado, la función debe retornar:

```
def sumar(a, b):  
    return a + b
```

13.3 Retornar demasiado pronto

```
def contar_pares(lista):  
    contador = 0  
  
    for n in lista:  
        if n % 2 == 0:  
            contador += 1  
        return contador # Error: está dentro del ciclo
```

El return debe ir después de terminar el recorrido:

```
def contar_pares(lista):  
    contador = 0  
  
    for n in lista:  
        if n % 2 == 0:  
            contador += 1  
  
    return contador
```

13.4 Confundir parámetro con variable externa

```
numero = 10  
  
def doble(numero):  
    return numero * 2  
  
print(doble(5)) # 10
```

El parámetro numero dentro de la función recibe el argumento enviado. En este caso, recibe 5.

13.5 Modificar listas sin intención

Las listas son mutables. Si una función modifica una lista recibida como parámetro, el cambio puede permanecer fuera de la función.

```
def agregar_cero(lista):  
    lista.append(0)  
  
datos = [1, 2, 3]  
agregar_cero(datos)  
  
print(datos) # [1, 2, 3, 0]
```

Esto no siempre es un error, pero debe hacerse de manera consciente.

14. Ejercicios propuestos

14.1 Funciones básicas

- Crear una función que reciba un número y retorne su triple.
- Crear una función que reciba dos números y retorne el mayor.
- Crear una función que reciba una nota y retorne True si es aprobada.
- Crear una función que reciba un número y retorne True si es par.

14.2 Funciones con listas

- Crear una función que cuente cuántos números positivos hay en una lista.
- Crear una función que retorne la suma de los números pares de una lista.
- Crear una función que retorne una lista con los valores mayores que 10.
- Crear una función que determine si una lista está ordenada de menor a mayor.

14.3 Funciones con cadenas

- Crear una función que cuente vocales en una cadena.
- Crear una función que retorne una cadena invertida.
- Crear una función que determine si una palabra es palíndroma.
- Crear una función que cuente cuántas veces aparece una letra específica.

14.4 Funciones con matrices

- Crear una función que sume todos los elementos de una matriz.
- Crear una función que cuente cuántos elementos negativos hay en una matriz.
- Crear una función que retorne el mayor elemento de una matriz.
- Crear una función que calcule la suma de una fila dada.

14.5 Ejercicio integrador

Diseñe un programa usando funciones y entripoint que:

- Reciba una matriz de notas.
- Cuente cuántas notas son aprobadas.
- Calcule el promedio general.
- Determine la mayor nota.
- Organice el flujo principal usando main() e if __name__ == '__main__'.

15. Checklist de estudio

- ¿Sé explicar qué es una función?
- ¿Diferencio parámetro y argumento?
- ¿Entiendo la diferencia entre print y return?
- ¿Sé hacer prueba de escritorio de una función?
- ¿Uso funciones auxiliares para separar condiciones o procesos?
- ¿Sé recorrer listas y matrices dentro de funciones?
- ¿Entiendo qué hace el entripoint?

- ¿Puedo organizar un programa usando main()?
- ¿Evito que una función haga demasiadas cosas?
- ¿Uso nombres claros y descriptivos?

16. Resumen final

Concepto	Idea clave	Ejemplo
Función	Bloque reutilizable con nombre.	<code>def sumar(a,b)</code>
Parámetro	Variable definida en la función.	<code>a, b</code>
Argumento	Valor enviado al llamar.	<code>sumar(2,3)</code>
return	Devuelve un resultado.	<code>return a+b</code>
main()	Coordina el programa.	<code>def main():</code>
entrypoint	Controla si el archivo se ejecuta como principal.	<code>if __name__ == '__main__':</code>

Frase final: programar con funciones no consiste solo en dividir código; consiste en dividir responsabilidades.